

Comprehensive Playwright Q&A with Code Examples

Playwright Automation Framework - Questions & Answers

1. What distinguishes Playwright from other end-to-end testing frameworks like Selenium or Cypress?

- Modern Architecture: Built for modern web apps, supports SPAs, shadow DOM, and iframes.
- Cross-Browser Support: Automates Chromium, Firefox, and WebKit using a single API.
- Auto-Waiting: Reduces flakiness by waiting for elements automatically.
- Network Interception: Built-in request/response mocking.
- Parallel Execution: Native support using browser contexts.
- Headless by Default: Optimized for CI/CD environments.

2. Explain Playwright's architecture—how does it achieve cross-browser automation?

- Uses a single API layer communicating via WebSocket.
- Bundles browser binaries (Chromium, Firefox, WebKit).
- Controls browsers using native DevTools protocols.
- Browser Contexts allow isolated sessions for parallel tests.

3. Which languages does Playwright support, and what are the trade-offs of each?

- JavaScript/TypeScript: Fast updates, best for Node.js teams.
- Python: Great for data-driven testing, fewer integrations.
- Java: Ideal for enterprise environments.
- .NET (C#): Good for Microsoft stack users.

Trade-offs: JS gets new features first; others lag slightly.

4. How does Playwright handle headless vs. headful browser modes?

- Headless Mode: Faster, ideal for CI/CD.
- Headful Mode: Shows UI, useful for debugging.

Example: `browser.launch({ headless: false })`

5. Describe Playwright's installation process and its bundled browser dependencies.

- Install via npm: `npm install @playwright/test`
- Run: `npx playwright install`
- Downloads Chromium, Firefox, WebKit binaries; no external drivers needed.

6. What built-in features does Playwright offer for test retries and flakiness mitigation?

- Auto-Waiting for elements.
- Retries: Configure in `playwright.config.ts` (`retries: 2`).
- Timeouts per action/test.
- Trace Viewer for debugging flaky tests.

7. How does Playwright ensure consistent behavior across Chromium, Firefox, and WebKit?

- Maintains browser-specific drivers internally.
- Normalizes APIs for similar behavior.
- Cross-browser test runner with projects configuration.

8. Explain Playwright's tracing feature and its benefits for debugging.

Example:

```
# Enable tracing
await context.tracing.start({ 'screenshots': True, 'snapshots': True })
await context.tracing.stop({ 'path': 'trace.zip' })
```

- Records screenshots, DOM snapshots, network logs.

- Enable tracing: `context.tracing.start({ screenshots: true })`.
- View with: `npx playwright show-trace trace.zip`.

9. What are Playwright “contexts” and how do they facilitate isolated test sessions?

Example:

```
# Create a new browser context
context = await browser.newContext()
page = await context.newPage()
```

- Browser Context = Independent session with its own cookies/storage.
- Enables parallel tests without multiple browsers.

Example: `browser.newContext()`.

10. Describe a scenario where Playwright’s multi-page support is critical.

- Example: Testing OAuth login or payment flow that opens a new tab.
- Use `context.waitForEvent('page')` to handle new tabs.

Playwright Setup & Configuration - Questions & Answers

11. Walk me through setting up a new Playwright project in Node.js from scratch.

- Initialize project: `npm init -y`
- Install Playwright: `npm install @playwright/test`
- Install browsers: `npx playwright install`
- Create test file: `example.spec.js`
- Run tests: `npx playwright test`

12. How do you configure Playwright in a non-JavaScript language (e.g., Python or C#)?

- Python: `pip install playwright` && `playwright install`

- C#: Use NuGet package Microsoft.Playwright
- Initialize: playwright codegen or create config files manually.

13. Where does Playwright store its browser binaries, and how can you customize that path?

- Default path: ~/.cache/ms-playwright
- Customize using PLAYWRIGHT_BROWSERS_PATH environment variable.

14. What command generates Playwright's initial test scaffold, and what does it include?

- Command: npx playwright init
- Includes: Example tests, config file, recommended folder structure.

15. How do you install only a subset of browsers (e.g., Chromium only) to save space?

- Command: npx playwright install chromium
- Similarly for firefox or webkit.

16. Explain how to integrate Playwright into an existing monorepo or workspace.

- Install Playwright in the specific package folder.
- Share config via root playwright.config.ts.
- Use workspaces for dependency management.

17. How can you pin Playwright to a specific browser version?

- Use PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=true and manually install desired version.
- Or specify browser executable path in config.

18. What environment variables or config files can you use to customize test runs?

- PLAYWRIGHT_BROWSERS_PATH: Custom browser path.
- PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD: Skip download.
- Config file: playwright.config.ts for retries, timeouts, projects.

19. How do you upgrade Playwright to the latest version with minimal breaking changes?

- Run: `npm install @playwright/test@latest`
- Check release notes for breaking changes.
- Update config accordingly.

20. What troubleshooting steps would you take if “npx playwright install” fails?

- Check internet connectivity.
- Verify permissions for cache folder.
- Use PLAYWRIGHT_BROWSERS_PATH to set custom path.
- Run with `--verbose` for logs.

Playwright Selectors & Element Handling - Questions & Answers

21. Compare CSS, XPath, text, role, and data-test selectors in Playwright—when to use each?

- CSS: Fast, native to browsers; best for stable class/id attributes.
- XPath: Useful for complex DOM traversal; less readable and slower.
- Text: `getByText()` for visible text; good for user-facing content.
- Role: `getByRole()` for accessibility-based selection; improves resilience.
- data-test: Custom attributes for test stability; recommended for dynamic apps.

22. How does Playwright’s `getByRole()` selector improve test resilience?

- Uses ARIA roles and accessibility tree, independent of DOM changes.

- Works across browsers and frameworks; less brittle than CSS/XPath.

23. Explain how to wait for an element to become attached, visible, enabled, or stable.

- Use `locator.waitFor()` or `expect(locator).toBeVisible()`.
- States: attached (in DOM), visible (rendered), enabled (interactive), stable (not animating).

24. What's the difference between `page.$()`, `page.$$()`, and `page.locator()`?

Example:

```
# Using locator for auto-waiting
button = page.locator('text=Submit')
await button.click()
```

- `page.$()`: Returns first matching element handle; one-time query.
- `page.$$()`: Returns array of element handles; one-time query.
- `page.locator()`: Persistent reference; supports auto-waiting and retries.

25. How do you handle dynamic elements whose selectors change at runtime?

- Use text or role-based selectors.
- Combine with regex or partial matches.
- Prefer data-test attributes for stability.

26. Describe how to chain locator filters (e.g., `.filter()`, `.nth()`).

Example:

```
# Using locator for auto-waiting
button = page.locator('text=Submit')
await button.click()
```

- Example: `page.locator('div').filter({ hasText: 'Submit' }).nth(0).click()`.

- Filters refine selection by text, visibility, or nested locators.

27. How would you click an element inside a shadow DOM or iframe?

- Shadow DOM: Use `locator('css-selector').shadowRoot().locator('inner-selector')`.
- iframe: Use `frameLocator('iframe-selector').locator('element').click()`.

28. Explain built-in auto-waiting: which actions implicitly wait and what do they wait for?

- Actions like `click()`, `fill()`, `type()` wait for element to be attached, visible, stable, and enabled.
- Reduces need for manual waits.

29. How can you assert an element's CSS property or computed style?

- Use `expect(locator).toHaveCSS('property', 'value')`.
- Example: `expect(button).toHaveCSS('background-color', 'rgb(0, 0, 255)')`.

30. What strategy would you use to locate elements in a highly data-driven, dynamic web app?

- Use data-test attributes for selectors.
- Combine role-based and text-based locators for resilience.
- Avoid brittle CSS/XPath tied to layout.

Playwright Test Structuring & Debugging - Questions & Answers

31. How do you structure tests using fixtures, hooks, and test suites in Playwright Test Runner?

- Use `test.describe()` to group related tests into suites.
- Define fixtures for reusable setup/teardown logic.
- Hooks like `beforeAll` and `afterAll` for suite-level setup/cleanup.

32. Describe the purpose of `beforeAll`, `beforeEach`, `afterEach`, and `afterAll`.

- `beforeAll`: Runs once before all tests in a suite (e.g., global setup).

- beforeEach: Runs before every test (e.g., fresh page context).
- afterEach: Cleans up after each test.
- afterAll: Runs once after all tests (e.g., close browser).

33. What best practices ensure test isolation and avoid state leakage between tests?

- Use new browser context for each test.
- Avoid sharing mutable state across tests.
- Clear cookies/storage between tests.

34. How do you share common setup logic across multiple test files?

- Create custom fixtures in a helper file.
- Use Playwright's test.extend() to share logic globally.
- Import shared utilities in test files.

35. Explain how retries and timeouts work at the test and suite level.

- Configure retries in playwright.config.ts (e.g., retries: 2).
- Set timeouts per test using test.setTimeout() or globally in config.

36. How would you parameterize a test to run with multiple input sets?

- Use test.describe() with loops or test() calls for each input.
- Or use test.step() with dynamic data arrays.

37. What are the trade-offs between using test.describe.parallel vs. sequential execution?

Example:

```
# Run tests in parallel using projects
projects = [
  { 'name': 'chromium', 'use': { 'browserName': 'chromium' } },
  { 'name': 'firefox', 'use': { 'browserName': 'firefox' } }
]
```

- Parallel: Faster execution, but requires strict isolation.
- Sequential: Safer for tests with shared resources, but slower.

38. How do you write custom matchers or extend Playwright's assertion library?

- Use `expect.extend()` to add custom matchers.
- Example: `expect.extend({ toBeWithinRange(received, floor, ceiling) { ... } })`.

39. Explain how to capture and attach screenshots and videos upon test failures.

- Enable screenshot/video in config: use: `{ screenshot: 'only-on-failure', video: 'retain-on-failure' }`.
- Access artifacts via `test.info().attachments`.

40. How do you debug flaky tests—what diagnostic information do you collect first?

- Collect trace files using `tracing.start()`.
- Review logs, screenshots, and videos.
- Check network requests and console errors.

Playwright Navigation & Interaction - Questions & Answers

41. How do you navigate to a URL and wait for specific network or load events?

Example:

- ```
Navigate to a URL and wait for load
await page.goto('https://example.com', { 'waitUntil': 'load' })
```
- Use `page.goto(url, { waitUntil: 'networkidle' })` for network idle.
  - Options: 'load', 'domcontentloaded', 'networkidle'.
  - Example: `await page.goto('https://example.com', { waitUntil: 'load' })`.

#### 42. Describe how to handle multiple tabs or popup windows in a test.

- Listen for 'page' event on context: `context.waitForEvent('page')`.
- Example: `const [newPage] = await Promise.all([context.waitForEvent('page'), page.click('selector')])`.

#### 43. How do you switch context into an iframe and interact with its contents?

*Example:*

---

```
Create a new browser context
context = await browser.newContext()
page = await context.newPage()
```

- Use `frameLocator('iframe-selector').locator('element').click()`.
- Example: `await page.frameLocator('#myFrame').locator('button').click()`.

#### 44. Explain Playwright's `page.waitForNavigation()` vs. `locator.waitFor()`.

*Example:*

---

```
Using locator for auto-waiting
button = page.locator('text=Submit')
await button.click()
```

- `page.waitForNavigation()`: Waits for page-level navigation events.
- `locator.waitFor()`: Waits for element state (attached, visible, etc.).

#### 45. How would you test a multi-step redirect flow across different domains?

- Use `page.waitForNavigation()` after each step.
- Validate final URL with `expect(page).toHaveURL()`.
- Handle cross-domain cookies via browser context.

#### 46. What are the pitfalls of using `page.reload()` or `page.goto()` repeatedly in tests?

- Can cause flakiness if network is slow.
- May reset state unexpectedly.
- Prefer navigation only when necessary.

#### 47. How do you handle file downloads and uploads via Playwright?

*Example:*

---

```
Upload a file
await page.locator('input[type=file]').setInputFiles('file.txt')
```

*Example:*

---

```
Handle file download
download = await page.waitForEvent('download')
await download.saveAs('downloaded.txt')
```

- Download: Use `page.waitForEvent('download')` and `download.saveAs(path)`.
- Upload: Use `input[type=file]` and `locator.setInputFiles('file-path')`.

#### 48. Explain how to intercept and handle dialogs (alerts, confirms, prompts) on a page.

- Listen for dialog event: `page.on('dialog', async dialog => { await dialog.accept(); })`.
- Can dismiss or provide input for prompts.

#### 49. How can you expose functions from the test into the page's JavaScript context?

*Example:*

---

```
Create a new browser context
context = await browser.newContext()
page = await context.newPage()
```

- Use `page.exposeFunction('funcName', callback)`.

- Example: `await page.exposeFunction('getData', () => 'test-data')`.

## **50. Describe a strategy for testing web apps that open complex modal dialogs or overlays.**

- Use locator chaining with `.filter()` for modal content.
- Wait for modal visibility using `expect(locator).toBeVisible()`.
- Interact with elements inside modal using scoped locators.

Playwright Network Interception & Mocking - Questions & Answers

## **51. How do you intercept network requests and modify responses on the fly?**

- Use `page.route()` to intercept requests.
- Modify response using `route.fulfill()` with custom body, headers, or status.

## **52. Explain the difference between `route.fulfill()`, `route.abort()`, and `route.continue()`.**

- `route.fulfill()`: Sends a mocked response to the request.
- `route.abort()`: Cancels the request entirely.
- `route.continue()`: Lets the request proceed, optionally modifying headers or payload.

## **53. How can you simulate slow network conditions or offline mode?**

- Use `context.setOffline(true)` for offline mode.
- Simulate slow network via `context.setDefaultTimeout()` or emulate network conditions using browser devtools protocol.

## **54. What's the recommended approach to mock API responses for deterministic tests?**

- Use `page.route()` with `route.fulfill()` and static JSON fixtures.
- Ensures tests are independent of backend availability.

### **55. How do you record and replay real API traffic in Playwright?**

- Capture network logs using tracing or HAR recording.
- Replay using `page.routeFromHAR()` for deterministic playback.

### **56. Describe how to verify request payloads and headers in tests.**

- Use `page.on('request')` to inspect `request.url()`, `request.postData()`, and `request.headers()`.
- Assert expected values using Playwright's expect API.

### **57. How do you stub only certain endpoints while letting others pass through?**

- Use conditional logic in `page.route()` to match specific URLs.
- Call `route.continue()` for non-matching requests.

### **58. Explain how to handle authentication tokens or cookies when mocking.**

- Include tokens in mocked response headers.
- Use `context.addCookies()` for session cookies before tests.

### **59. What challenges arise when testing WebSocket or SSE connections, and how do you intercept them?**

- WebSocket/SSE are persistent connections, harder to mock.
- Use `page.on('websocket')` or simulate messages via custom scripts.

### **60. How do you ensure your mock data stays in sync with evolving backend contracts?**

- Maintain shared schema or contract tests.
- Automate validation against API specs (e.g., OpenAPI).

## 61. How do you configure Playwright to run tests across Chromium, Firefox, and WebKit?

- Use projects in playwright.config.ts:

```
projects: [
 { name: 'chromium', use: { browserName: 'chromium' } },
 { name: 'firefox', use: { browserName: 'firefox' } },
 { name: 'webkit', use: { browserName: 'webkit' } }
]
```

- Run: `npx playwright test`

## 62. What options are available to emulate mobile devices and set viewport sizes?

### *Example:*

---

```
Emulate mobile device
from playwright.devices import devices
use = { **devices['iPhone 12'] }
```

- Use device descriptors from playwright.devices (e.g., iPhone 12).
- Example: `use: { ...devices['iPhone 12'] }`.
- Or set viewport manually: `use: { viewport: { width: 375, height: 812 } }`.

## 63. How do you test geolocation- or device-orientation-sensitive features?

### *Example:*

---

```
Emulate mobile device
from playwright.devices import devices
use = { **devices['iPhone 12'] }
```

- Enable permissions: `use: { permissions: ['geolocation'] }`.
- Set geolocation: `context.setGeolocation({ latitude: 37.7749, longitude: -122.4194 })`.
- Device orientation can be simulated via browser devtools protocol.

## 64. Explain how to capture and compare screenshots for visual regression across browsers.

- Use `page.screenshot()` to capture images.
- Compare using `pixelmatch` or Playwright's `toHaveScreenshot()` matcher.
- Run across all projects for cross-browser validation.

## 65. What considerations apply when running tests on iOS or Android emulators vs. real devices?

*Example:*

---

```
Emulate mobile device
from playwright.devices import devices
use = { **devices['iPhone 12'] }
```

- Emulators may not replicate hardware performance or sensors accurately.
- Real devices provide true network and rendering behavior.
- Use cloud device farms for broader coverage.

## 66. How can you integrate Playwright tests with a cloud-based device farm?

*Example:*

---

```
Emulate mobile device
from playwright.devices import devices
use = { **devices['iPhone 12'] }
```

- Use services like BrowserStack or Sauce Labs.
- Configure remote browser endpoints via `connectOverCDP` or `wsEndpoint`.

## 67. Describe the process to record video for each browser session and attach it to reports.

- Enable video in config: `use: { video: 'on' or 'retain-on-failure' }`.
- Access video via `test.info().attachments` after test completion.

## 68. How do you detect and handle browser-specific quirks or CSS prefixes in tests?

- Use conditional logic based on browserName in test.info().project.name.
- Validate computed styles using toHaveCSS().

## 69. Explain how to parameterize your test matrix to include browser, device, and locale.

*Example:*

---

```
Emulate mobile device
from playwright.devices import devices
use = { **devices['iPhone 12'] }
```

- Define multiple projects in config with different use settings (browserName, locale, device).
- Example: use: { locale: 'fr-FR', ...devices['Pixel 5'] }.

## 70. What metrics would you gather to compare performance across browsers?

- Page load time, first contentful paint, network requests count.
- Use page.metrics() or browser devtools protocol for detailed performance data.

Playwright CI/CD & Pipeline Integration - Questions & Answers

## 71. How do you run Playwright tests in GitHub Actions, and what key config steps are needed?

- Use official Playwright GitHub Action or custom workflow.
- Install dependencies: npm ci && npx playwright install.
- Add job steps for caching, running tests, and uploading artifacts.

## 72. Explain how to distribute tests across multiple machines or containers for parallelism.

*Example:*

---

```
Run tests in parallel using projects
projects = [
 { 'name': 'chromium', 'use': { 'browserName': 'chromium' } },
 { 'name': 'firefox', 'use': { 'browserName': 'firefox' } }
]
```

- Use sharding with --shard option: `npx playwright test --shard=1/3`.
- Configure multiple runners in CI matrix strategy.

### **73. What environment variables control Playwright's behavior in CI (e.g., CI, DEBUG)?**

- `CI=true`: Enables CI mode (disables headful by default).
- `DEBUG=pw:verbose` for detailed logs.
- `PLAYWRIGHT_BROWSERS_PATH` for custom browser path.

### **74. How can you generate and store HTML or JUnit reports in a CI pipeline?**

- Configure reporter in `playwright.config.ts`: `reporters: [['html'], ['junit']]`.
- Upload report folder as artifact in CI job.

### **75. Describe how to configure retries only on CI, but not on local development runs.**

- Use conditional logic in config: `retries: process.env.CI ? 2 : 0`.

### **76. What are the best practices for caching browser binaries and node modules in CI?**

- Cache `~/.cache/ms-playwright` and `node_modules`.
- Use `actions/cache` in GitHub Actions for efficient builds.

### **77. How do you set up a custom reporter (e.g., Slack, TestRail) in your pipeline?**

*Example:*

---

```
Configure HTML and JUnit reporters
reporters: [['html'], ['junit']]
```

- Implement custom reporter class extending Reporter API.
- Configure in playwright.config.ts: reporters: [['custom-reporter']].

## 78. Explain how to detect and fail fast on critical test failures to save CI minutes.

- Use --max-failures flag: `npx playwright test --max-failures=1`.
- Stops execution after first failure.

## 79. How do you integrate Playwright tests into Docker-based build flows?

- Add Playwright dependencies in Dockerfile.
- Install browsers via `npx playwright install`.
- Run tests inside container with proper volume mounts.

## 80. What strategies help diagnose CI-only failures vs. local-only failures?

- Collect trace, screenshots, and videos in CI.
- Compare environment variables and browser versions.
- Use DEBUG logs and run tests in headful mode locally.

Playwright Reporting & Debugging - Questions & Answers

## 81. What built-in reporters does Playwright provide, and when would you choose each?

*Example:*

---

```
Configure HTML and JUnit reporters
reporters: [['html'], ['junit']]
```

- Built-in reporters: list, dot, line, json, junit, html.
- Use html for visual reports, junit for CI integration, json for custom processing.

## 82. How do you generate an HTML trace and use the Trace Viewer to debug a test?

- Enable tracing: `context.tracing.start({ screenshots: true, snapshots: true })`.
- Stop and save: `context.tracing.stop({ path: 'trace.zip' })`.
- View trace: `npx playwright show-trace trace.zip`.

## 83. Explain how to attach console logs, network logs, and request/response bodies to reports.

- Listen to `page.on('console')` and `page.on('request')` events.
- Collect logs and attach via `test.info().attachments` in custom reporter.

## 84. How can you configure Playwright to take a screenshot on every action or step?

- Use screenshot option in config: `use: { screenshot: 'on' }`.
- Or manually capture in test steps using `page.screenshot()`.

## 85. Describe how to write a custom reporter plugin for your own dashboard.

### *Example:*

---

```
Configure HTML and JUnit reporters
reporters: [['html'], ['junit']]
```

- Implement Reporter interface with `onBegin`, `onTestEnd`, `onEnd` methods.
- Export as module and configure in `playwright.config.ts`.

## 86. What tools help you step through a paused test and inspect page state interactively?

- Use `npx playwright test --debug` for inspector mode.
- Playwright Inspector allows stepping through actions and DOM inspection.

### **87. How do you debug tests in CI where you can't launch a headed browser?**

- Collect trace, screenshots, and videos.
- Use headless mode with DEBUG logs enabled.
- Review artifacts after run.

### **88. Explain the usage of DEBUG=pw:api or other debug environment flags.**

- DEBUG=pw:api logs Playwright API calls.
- Other flags: pw:browser, pw:protocol for deeper debugging.

### **89. How can you integrate Playwright's output with external test management systems?**

- Use junit or json reporters for structured output.
- Parse results and push to systems like TestRail or Zephyr via API.

### **90. What metrics would you collect from test runs to measure suite health over time?**

- Test duration, pass/fail rates, retry counts.
- Flakiness trends and resource usage metrics.

Playwright Advanced Integration & Best Practices - Questions & Answers

### **91. Compare Playwright Test Runner vs. using Playwright with Jest or Mocha—pros and cons?**

- Playwright Test Runner: Built-in parallelism, auto-waiting, rich reporters, first-class Playwright support.
- Jest/Mocha: Familiar for existing projects, larger ecosystem, but requires extra setup for Playwright features.

### **92. What community plugins or extensions have you used to enhance Playwright's functionality?**

- Examples: allure-playwright for reporting, eslint-plugin-playwright for linting, cucumber-js for BDD integration.

### **93. How would you integrate Playwright into a larger BDD framework like Cucumber?**

- Use cucumber-js with Playwright steps.
- Map Gherkin steps to Playwright actions via step definitions.

### **94. Explain how Playwright's experimental features (e.g., web component testing) might fit your needs.**

- Experimental features allow testing shadow DOM and custom elements.
- Useful for modern frameworks like Lit or Stencil.

### **95. How can you leverage Playwright's API to build a custom test orchestration tool?**

- Use Playwright's `browserType.launch()` and context APIs to manage sessions.
- Integrate with custom schedulers or CI pipelines for distributed runs.

### **96. Describe how to extend Playwright's `playwright.config.js` with TypeScript or advanced logic.**

- Use `playwright.config.ts` for type safety.
- Add dynamic logic based on environment variables or CLI args.

### **97. What patterns help you organize and share helper functions or page-object modules?**

- Use Page Object Model (POM) for maintainability.
- Store reusable locators and actions in separate classes or modules.

### **98. How do you keep your test suite maintainable as the application under test grows?**

- Modularize tests and helpers.
- Use fixtures for common setup.
- Regularly refactor and remove flaky tests.

**99. Explain the role of Playwright in performance testing or accessibility auditing.**

- Performance: Measure metrics via `page.metrics()` or DevTools protocol.
- Accessibility: Integrate axe-core with Playwright for audits.

**100. What are emerging trends in Playwright's roadmap that you're excited about?**

- Component testing support.
- Enhanced mobile emulation and real-device integration.
- Better debugging and cloud execution features.